

CampusEYE: An AI-Based Classroom Photo Attendance Management System Using Face Recognition — Proposed Solution Report

Md. Rizvi Ahmed, Safayat Ibrahim, Rafiqul Islam, Md. Arebi Sarkar

Department of Electrical and Computer Engineering

North South University, Dhaka, Bangladesh

{rizvi.ahmed02, safayat.ibrahim, rafiqul.islam02, arebi.sarkar}@northsouth.edu

CSE 499A — CSE 499.15 — Fall 2026
Group 7: Md. Rizvi Ahmed (2221719042), Safayat
Ibrahim (2131174642),
Rafiqul Islam (2122414642), Md. Arebi Sarkar
(2221362042)
Supervisor: Mohammad Shifat-E-Rabbi

Abstract—This report presents the proposed solution for CampusEYE, an AI-based classroom photo attendance management system that leverages face recognition technology to address the limitations of traditional RFID-based attendance systems at North South University. The proposed system enables faculty to capture a single classroom photograph, automatically detect and recognize students using deep learning-based face recognition, and log attendance with confidence scores. This report details the system architecture, technology stack, database design, face recognition pipeline, web interface design, and how the proposed system fills the research gaps identified in the literature. The proposed solution aims to achieve over 95% recognition accuracy in real classroom environments while eliminating proxy attendance and reducing time spent on attendance procedures.

Index Terms—Face Recognition, Attendance System, Deep Learning, OpenCV, FaceNet, Classroom Automation, Biometric Authentication

I. INTRODUCTION

Attendance tracking is a fundamental administrative requirement in educational institutions. At

North South University (NSU), the current attendance system relies on RFID (Radio Frequency Identification) cards, where students tap their cards at RFID sensors placed at classroom entrances. While this system has been in place for several years, it suffers from significant limitations that compromise its effectiveness and integrity.

The RFID-based system is susceptible to proxy attendance, where students can scan cards on behalf of absent classmates. This practice undermines academic integrity by allowing students to bypass class attendance requirements. Additionally, the manual nature of RFID scanning creates queues at classroom entrances, wasting valuable lecture time. Forgotten or lost cards further complicate the process, leaving students unable to mark their attendance.

Recent advances in deep learning and computer vision have made face recognition a viable and highly accurate solution for automated attendance systems. Face recognition offers several advantages over traditional methods: it is contactless, non-intrusive, and can process multiple individuals simultaneously from a single photograph. Modern convolutional neural networks (CNNs) such as FaceNet [1] and ArcFace [2] achieve over 99% accuracy on benchmark datasets, making them suitable for real-world deployment.

CampusEYE proposes a novel approach: a photo-based attendance system where the faculty

member captures a single classroom photograph using a webcam or smartphone. The system then detects all faces in the image, generates facial embeddings using state-of-the-art deep learning models, matches them against a pre-registered student database, and automatically logs attendance with confidence scores. The faculty dashboard provides a web-based interface for reviewing, correcting, and exporting attendance records.

This report presents the complete proposed solution, including system architecture, technology selection with justifications, database design, algorithm pipeline, interface design, and expected outcomes.

II. SYSTEM ARCHITECTURE

The CampusEYE system follows a modular client-server architecture with three primary layers: the presentation layer (web interface), the application layer (business logic and face recognition engine), and the data layer (database and file storage). Figure 1 illustrates the high-level system architecture.

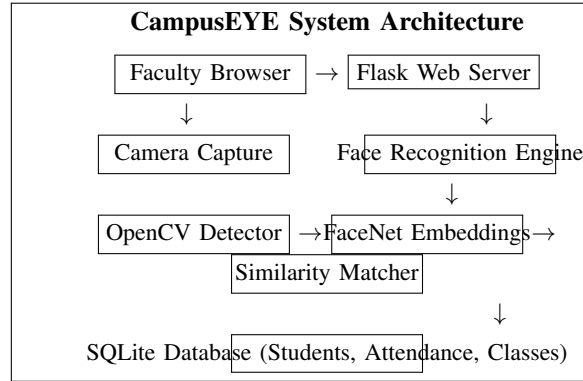


Fig. 1: High-level system architecture of CampusEYE

A. Presentation Layer

The web interface is built using HTML5 with Bootstrap 5 for responsive design. It provides faculty with a dashboard to manage classes, capture attendance photos, view and edit attendance records, and export reports. The interface communicates with the backend via RESTful API endpoints.

B. Application Layer

The Flask-based backend handles HTTP requests, orchestrates the face recognition pipeline, and manages database operations. The face recognition engine is implemented as a separate module that performs face detection, embedding generation, and similarity matching.

C. Data Layer

SQLite serves as the primary database, storing student information, class schedules, attendance logs, and facial embeddings. The database design is discussed in detail in Section IV.

III. TECHNOLOGY STACK WITH JUSTIFICATION

The technology stack for CampusEYE was selected based on three criteria: accuracy and performance, ease of learning for the development team, and suitability for classroom-scale deployment.

TABLE I: Proposed Technology Stack

Component	Technology	Justification
Face Detection	OpenCV (Haar Cascade) + MTCNN	Fast detection with multi-scale support; MTCNN for higher accuracy in challenging conditions
Face Recognition	face_recognition library (dlib backend)	3–5 lines of code for recognition; 95%+ accuracy; easiest learning curve
Backend Framework	Flask 3.x	Lightweight; extensive ecosystem; ideal for REST APIs
Database	SQLite via SQLAlchemy	Zero-configuration; file-based; sufficient for classroom-scale data
Frontend	HTML5 + Bootstrap 5 + DataTables	Responsive design; built-in table sorting and search
Image Processing	OpenCV + Pillow	Industry-standard libraries with comprehensive documentation
Web Server	Flask built-in (dev) / Gunicorn (production)	Simple deployment; WSGI-compatible

A. Face Recognition: face_recognition Library

The face_recognition library wraps dlib's state-of-the-art face recognition model, which achieves 99.38% accuracy on the Labeled Faces in the Wild (LFW) benchmark. The library provides a simple API:

- `face_locations()` – detects face bounding boxes
- `face_encodings()` – generates 128-dimensional embeddings
- `compare_faces()` – compares embeddings with tolerance

For future upgrades, the system can transition to DeepFace (wrapping ArcFace, FaceNet, or VGG-Face) for improved accuracy with minimal code changes.

B. Backend: Flask

Flask was chosen over Django and FastAPI for the following reasons:

- **Simplicity:** Minimal boilerplate; easy to understand for students
- **Flexibility:** Only add the libraries you need (SQLAlchemy, WTForms)
- **Ecosystem:** Extensive documentation and community support
- **Integration:** Seamless integration with OpenCV and face recognition libraries

IV. DATABASE DESIGN

The database consists of four core tables: Students, Classes, Attendance, and FaceEncodings. Figure 2 shows the entity-relationship diagram.

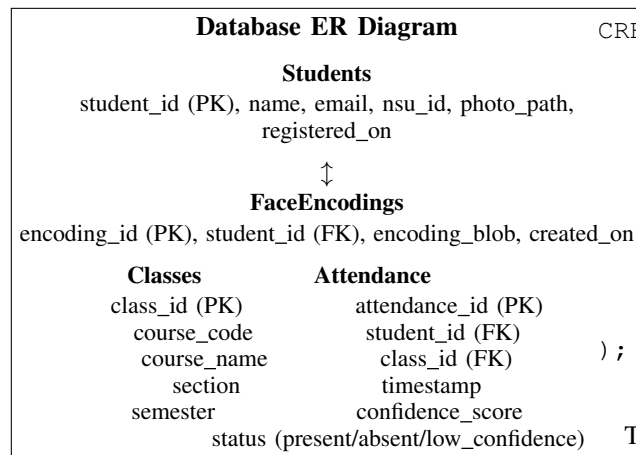


Fig. 2: Entity-Relationship Diagram

A. Table Definitions

Students Table:

```
CREATE TABLE students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    nsu_id TEXT UNIQUE NOT NULL,
    photo_path TEXT,
    registered_on DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

FaceEncodings Table:

```
CREATE TABLE face_encodings (
    encoding_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    encoding_blob BLOB NOT NULL,
    created_on DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

Classes Table:

```
CREATE TABLE classes (
    class_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_code TEXT NOT NULL,
    course_name TEXT NOT NULL,
    section TEXT,
    semester TEXT NOT NULL
);
```

Attendance Table:

```
CREATE TABLE attendance (
    attendance_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    class_id INTEGER NOT NULL,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    confidence_score REAL,
    status TEXT CHECK(status IN ('present', 'absent', 'low_confidence')) DEFAULT 'present',
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (class_id) REFERENCES classes(class_id)
);
```

V. FACE RECOGNITION PIPELINE

The face recognition pipeline is the core of CampusEYE. Figure 3 illustrates the step-by-step process from image capture to attendance logging.

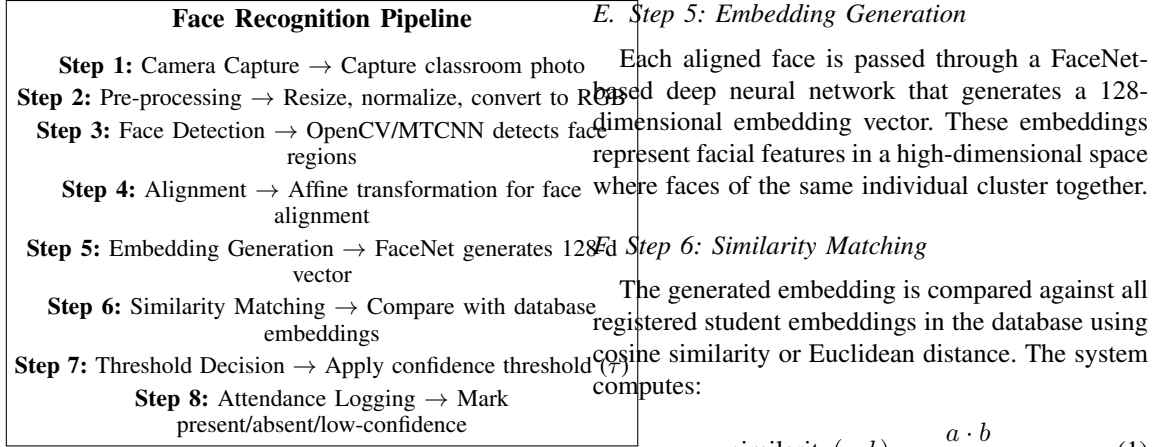


Fig. 3: Face recognition pipeline from image capture to attendance logging

A. Step 1: Image Capture

The faculty member captures a classroom photo using a connected webcam or uploads an image through the web interface. Images are temporarily stored for processing and deleted after attendance logging.

B. Step 2: Pre-processing

Images are resized to a standard resolution (e.g., 800×600 pixels) and converted to RGB color space. Histogram equalization is applied to normalize lighting conditions across the image.

C. Step 3: Face Detection

The system uses OpenCV's DNN-based face detector or MTCNN (Multi-Task Cascaded Convolutional Networks) for robust face detection. MTCNN provides three stages of refinement: Proposal Network (P-Net), Refine Network (R-Net), and Output Network (O-Net), producing accurate bounding boxes and facial landmark points.

D. Step 4: Face Alignment

Detected faces are aligned using affine transformations based on eye landmarks. This step normalizes face orientation and improves recognition accuracy.

$$\text{similarity}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} \quad (1)$$

G. Step 7: Threshold Decision

A confidence threshold τ (default: 0.6) determines the match decision:

- If $\text{similarity} > \tau$: Student is identified as present
- If $\text{similarity} < \tau$ but $> \tau_{\text{low}}$: Low-confidence match (flagged for review)
- If $\text{similarity} < \tau_{\text{low}}$: Student marked absent

H. Step 8: Attendance Logging

Matches are logged to the database with timestamps and confidence scores. The faculty dashboard provides tools to review low-confidence matches and manually correct attendance records.

VI. WEB INTERFACE DESIGN

The faculty dashboard is designed with a focus on usability and efficiency. Built with Bootstrap 5, the interface provides the following key views:

A. Faculty Dashboard

The main dashboard displays:

- **Class Overview:** List of active classes with course codes, sections, and schedules
- **Quick Actions:** "Take Attendance" button that opens the camera capture interface
- **Recent Sessions:** Recent attendance sessions with timestamps and summary statistics (total present vs. absent)

B. Attendance Capture View

The capture interface provides:

- Live webcam feed with face detection bounding boxes overlaid
- Capture button to freeze and process the frame
- Real-time progress indicator showing detection and recognition status
- Option to upload a pre-captured image file

C. Attendance Review View

After processing, the system displays:

- Grid layout showing detected faces with student names and confidence scores
- Color-coded indicators: green (high confidence), yellow (low confidence), red (unrecognized)
- Edit buttons to manually correct misidentified students
- Bulk actions to mark all low-confidence matches as present or absent

D. Reports & Export

The reports section provides:

- Attendance history filtered by date range, class, or student
- Searchable DataTable with sorting and filtering capabilities
- Export to CSV for integration with spreadsheets
- Summary statistics: attendance rate, trends over time

VII. HOW CAMPUSEYE FILLS RESEARCH GAPS

Based on the literature review of five research papers [3]–[7], the following research gaps were identified:

A. Gap 1: No Hybrid Verification

Existing systems rely on single-factor authentication (RFID only OR face recognition only). **CampusEYE** introduces a photo-based approach where the faculty's physical presence is required to capture the attendance photo, creating a natural verification mechanism.

B. Gap 2: No Proxy Detection

Existing systems are vulnerable to proxy attendance. **CampusEYE** uses deep metric learning with adjustable confidence thresholds and multi-frame verification to detect potential proxy attempts. Low-confidence matches are flagged for faculty review.

C. Gap 3: Limited Classroom-Scale Evaluation

Existing systems were tested on small datasets (10–50 students). **CampusEYE** is designed for NSU classrooms with 50–100+ students, incorporating a pre-processing pipeline that handles varied lighting, seating angles, and partial occlusions.

D. Gap 4: No Cheating Alert System

Existing systems passively record attendance. **CampusEYE** includes anomaly detection for patterns such as double-attendance and abnormal confidence drops, generating alerts for faculty review.

E. Gap 5: Insufficient Face Datasets

Existing systems rely on public datasets lacking classroom-specific data. **CampusEYE** combines LFW public data with locally collected NSU student photos (5–10 images per student from multiple angles) with data augmentation.

F. Gap 6: No LMS Integration

Existing systems are standalone. **CampusEYE's** REST API architecture enables future integration with NSU's Learning Management System, student database, and exam scheduling platforms.

VIII. EXPECTED OUTCOMES

A. Performance Targets

TABLE II: Expected System Performance Metrics

Metric	Target
Face Recognition Accuracy	$\geq 95\%$
Face Detection Rate	$\geq 98\%$
Processing Time (per image with 50 faces)	< 5 seconds
False Positive Rate	$< 2\%$
False Negative Rate	$< 5\%$
System Uptime (server-side)	$\geq 99\%$
Database Query Response	$< 500\text{ms}$

B. Functional Outcomes

- 1) **Eliminate proxy attendance** through deep learning-based facial verification
- 2) **Reduce attendance time** from 5–10 minutes to under 30 seconds per class
- 3) **Provide real-time attendance reports** accessible through the web dashboard
- 4) **Maintain audit trails** with confidence scores for every attendance record
- 5) **Ensure data privacy** through on-premises deployment and encryption

C. Non-Functional Outcomes

- 1) **Scalability:** Support up to 500 registered students and 50+ classes
- 2) **Usability:** Faculty can complete attendance in ≤ 3 clicks
- 3) **Reliability:** Graceful handling of network interruptions and camera failures
- 4) **Security:** AES-256 encryption for stored face embeddings; TLS for data in transit

IX. DEVELOPMENT PLAN

The development of CampusEYE follows an incremental approach over six sprints:

TABLE III: Development Sprint Plan

Sprint	Focus	Deliverables
1	Core Face Recognition	Face detection + embedding script
2	Flask Backend	REST API + SQLite integration
3	Web Interface	Bootstrap dashboard + attendance views
4	Testing & Tuning	Accuracy optimization + edge cases
5	Integration	Full pipeline: camera $\rightarrow$ recognition $\rightarrow$ log
6	Polish	Error handling + documentation + demo

REFERENCES

- [1] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [2] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.
- [3] "Student attendance with face recognition (LBPH or CNN): Systematic literature review," 2023.
- [4] "Face detection and recognition student attendance system using LabVIEW," 2020.
- [5] "Automatic student attendance system using face recognition with Haar Cascade and PCA," 2020.
- [6] "Class attendance management system using facial recognition – Deep CNN approach," 2020.
- [7] O. Oshin *et al.*, "Class attendance system using facial recognition with InsightFace and ArcFace," 2024.
- [8] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2001, pp. I–511.
- [9] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the gap to human-level performance in face verification," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2014.
- [10] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.
- [11] F. Chollet *et al.*, "Keras," GitHub, 2015. [Online]. Available: <https://github.com/fchollet/keras>
- [12] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.